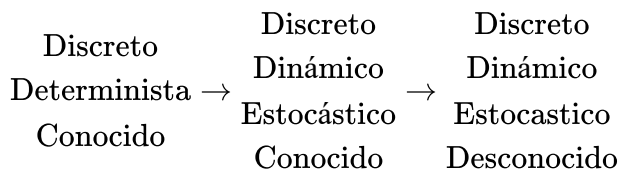


4. Juegos (Hasta aquí primer examen)

Planeación

Ir de un estado inicial a un estado terminal, con el menor costo.

Los planes pasan por 3 fases diferentes.



En otras palabras, el plan tiene las siguientes fases:

Busqueda Heurística → Markov's Decision Process → Árboles de decisión

Vamos a hacer un juego genérico de dos jugadores:

```
class Juego2T2D(object):
    # Juego a jugar
    def acciones_legales(self, estado, jugador):
        # Acciones legales que un jugador puede tomar
        # Se distinguen los dos jugadores. 1 para el primero y -1 para el
segundo.
        # Código
        return conjunto_acciones

    def sucesor(self, estado, accion, jugador):
        # Da el proximo estado para tu jugador
        # Código
        return nuevo_estado

    def terminal(estado):
        # Código
        return bool

    def ganancia(self, estado):
        # Código
        return num
        # Devuelve positivo o negativo, dependiendo del jugador
```

En el caso del juego del gato sabemos que:

- Estados: $S = (s_0, \dots, s_8)$ y $S_i \in \{1, 0, -1\}$
- Acciones: $a \in \{0, \dots, 8\}$

```
class Gato(Juego2T2D):
    # s = [0, 1, 2, 3, 4, 5, 6, 7, 8]
    def acciones_legales(self, estado, jugador):
        return [i for i in range(9), if estado[i] == 0]
        # Regresa el numero si el tile del tablero esta desocupado
        # Alternativa es:
        # return [i for (i,s) in enumerate(estado) if s == 0]

    def sucesor(self, estado accion, jugador):
        s_n = estado[i] # Se almacena la posición en el tablero
        s_n[acción] = jugador # El tile es reclamado por el jugador con su id
        return s_n # Regresamos el tile reclamado

    def ganacia(self, estado):
        #if 0 not in estado: # Si 0 no esta en el vector estado
        #    return 0 # Entonces el juego se acaba

        # El codigo comentado arriba es medio redundante

        # Programamos las diagonales
        if ((s[0] == s[4] == s[8] == 1 != 0) or
            (s[2] == s[4] == s[6])) and s[4] != 0:
            return s[4]

        # Programamos lineas
        for i in range(3):
            if ((s[i] == s[i + 3] == s[s + 6]) or
                (s[3i] == s[3i + 1] == s[3i+2])) and s[3i] != 0:
                return s[3i]

        return 0 # Si no encontramos lineas
```

Como referencia, así se ve el tablero:

0	1	2
3	4	5
6	7	8

Como ejemplo, vamos a programar un juego humano vs AI:

```

gato = Gato()
humano = 1 # 1 a -1
s = [0] * 9 # Definimos estados
j = 1 # Jugador que empieza

actualiza_tablero(S)

for _ in range(9):
    if j == humano: # Si j es humano
        acciones = gato.acciones_legales(s, j) # Se toman las acciones del
        juego
        a = escoge_accion(acciones) # Te da las opciones y toma los escogido
    else: # Si j es el AI
        a = agente(gato, s, j) # El agente toma acción

    s = gato.sucesor(s, a, j) # Se calcula el cambio de estado
    actualiza_tablero(s) # Actualizamos visualmente el tablero
    j = -j # Se cambia de jugador

    # Checas si se hace linea y terminas el juego.
    if gato.terminal(s):
        break

actualiza_score(gato.ganancia(s)) # Sale quien ganó

```

Algoritmo Minimax

$$\begin{aligned}
 & \max(x, y) \\
 \min(x, y) &= -\max(-x, -y) \\
 & * \max(*x, *y)
 \end{aligned}$$

Hay un algoritmo que combina ambas funciones llamada *negamax*.

El algoritmo Alpha beta pruning es exactamente lo mismo que la búsqueda minimax. lo cual significa que siempre hacemos la búsqueda optima sin revisar todos los nodos.

- Teoría de la utilidad.
- Tablas de transposición.